

SOC stand simulation project

By:

- Galiev Arsen - a.galiev@innopolis.university
- Nguen Ilya-Linh - i.nguen@innopolis.university
- Zavadskii Peter - p.zavadskii@innopolis.university

Goal/Tasks of the Project

The main goal of the project, is to create a simulated work of protected by a SIEM and WAF network that would be attacked. We created the architecture below:



Therefore, we are able not only detect and prevent typical attacks as brute-force, malware installation and pwns via Wazuh SIEM, but detect and prevent some of them and additional attacks (e.g. injections) via IDS/IPS Suricata.

Thus, we have the following stack :

- 5 virtual machines. 3 Ubuntu, 1 Windows and 1 ParrotOS
- Wazuh (SIEM system)
- Suricata (IDS/IPS)
- Nginx (Load-balancer)

Configuration plan (Environment preparation)

1. Architecture creation, configuring private network using L3 router and starting components deploying.
2. Deploy nginx with custom config that will redirect all the requests to the required servers. Install Suricata and create rules for preventing XSS attacks.
3. Deploy Wazuh master and Wazuh agents on different virtual machines and create connection between them. Deploy JuiceShop application to test attacks.

Development of solution/tests as the PoC

1. We followed the execution plan and created a private network with all necessary components.



Difficulties faced, new skills acquired during the project

There were several difficulties related to Wazuh agent-server connection and Windows machine integration:

- We used following command to set up manager:

```
curl -sO https://packages.wazuh.com/4.11/wazuh-install.sh && sudo bash
./wazuh-install.sh -a
```

And probably since we used not the latest version, the agents configs files were not able to run agents from the box, so we had to remove some unused declarations.

- Config files rules managing

However, we finally coped with it by troubleshooting the machines.



Therefore, we increased our understanding in DevSecOps field.

Setup

Wazuh

As we stated above, we used a command to install Wazuh manager. After installation and machines availability check, we deployed agents:

- Windwos10

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.11/windows/wazuh-agent-
4.11.0-1.msi -OutFile $env:TMP\wazuh-agent.msi; msixec.exe /i
$env:TMP\wazuh-agent.msi /q WAZUH_MANAGER='192.168.31.152'
WAZUH_AGENT_NAME='Windows10'; Start-Service wazuhSvc

NET start WazuhSvc
```

- Ubuntu 24.04

```
sudo wget https://packages.wazuh.com/4.11/apt/pool/main/w/wazuh-
agent/wazuh-agent_4.11.0-1_amd64.deb && sudo WAZUH_MANAGER='192.168.31.152'
dpkg -i ./wazuh-agent_4.11.0-1_amd64.deb

sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

Since we had conflicts in agents and `ossec.conf` installed by them, we removed rules for *browser extensions, processes and groups*.

Config for Ubuntu: [configs/ubuntu/ossec.conf](#)

Config for Windows: [configs/windows/ossec.conf](#)

Suricata

To install Suricata we used the following guide: habr.com

```
sudo apt-get update
sudo apt-get install -y software-properties-common
sudo add-apt-repository ppa:oisf/suricata-stable
sudo apt-get update
sudo apt-get install -y suricata
suricata --build-info
sudo suricata -c /etc/suricata/suricata.yaml -i wlp0s20f3 # interface name
```

Nginx

Here how was the load-balancer configured: nginx.conf

Also, to simulate having DNS-server we used /etc/hosts files on other machines:

- Windows: system32/drivers/etc/hosts
- Linux (Parrot & Ubuntu): [/etc/hosts](https://etc/hosts)

PoC, Demonstration

Plan

We have 2 "client machines", one hosting vulnerable application. Our testing included:

- Attack Juice-Shop via XSSi.
- Ubuntu 24.04 machine attack via [hydra](#) as we did on lab.
- Windows 10 machine SMB pseudo-brute-force with [smbclient](#) and [hydra](#) on ParrotOS.

Juice Shop

Since we created a primitive [anti-XSS suricata rules](#), we made XSS attacks simulation (see the demo):

```
curl "http://juiceshop/q?=<iframe%20javascript%3D%22alert('')%2F%3E"
```

and got 405 HTTP response code.



Ubuntu check

Some host information after SCA. Here we filtered only high-severity vulnerabilities since ubuntu host is Arsen's personal workstation:



Logs work well:



Let's repeat the lab scenario with ssh brute force:



Again we see familiar attacker's IP, but now in log message.

Windows check

Windows Vulnerability detect:



As you can see on the video, we detected in Wazuh events that Parrot OS attacker's attempts to brute windows guest session password:



Some details and we know that attackers used Parrot OS, it's IP and other details:



Success

During the analysis it gave a thought Windows logs a bit better structured for SIEM normalization, comparing Linux syslog.

Conclusion

In conclusion, this project successfully demonstrated the implementation of a Security Operations Center (SOC) model utilizing open-source tools, including Suricata + Nginx for network intrusion detection and prevention, almost WAF simulation, and Wazuh for endpoint security. The integrated architecture, incorporating Nginx as a load balancer and JuiceShop as a vulnerable application, facilitates comprehensive testing of attack detection mechanisms, such as cross-site scripting (XSS), and incident response capabilities.

The system underscored the critical role of centralized event collection in enabling rapid threat identification and mitigation. Furthermore, this proof-of-concept serves as a foundational platform for advancing more sophisticated security scenarios, encompassing WAF, OpenCTI, SIEM-EDR combinations (Splunk + Wazuh) as cluster-based deployments.

GitHub

<https://github.com/quintet-sdr/demo-soc>

Demo video

<https://github.com/quintet-sdr/demo-soc/blob/main/recordings/GalievNguyenZavadskii-demo.mp4>

Tools used

Infra:

- Mi router
- Oracle VirtualBox Community
- VMWare Workstation Pro

Defense tools:

- Wazuh Managment deploy
- Suricata IDS
- Nginx

OS:

- Windows 10 Pro
- Ubuntu 24.04 LTS
- Parrot OS
- Docker
- OWASP Juice Shop

Attack tools:

- smbclient
- hydra
- curl
- nmap